

Localização Indoor Baseada em Imagens do Teto

Estimativa de posição em ambientes fechados por reconhecimento visual do teto auxiliado por GPS aproximado

Munif Gebara · Projeto de pesquisa · Junho de 2026

Resumo

A localização de pessoas e ativos em ambientes fechados é um problema relevante e ainda mal resolvido: o GPS degrada-se fortemente sob coberturas e lajes, e soluções baseadas em Wi-Fi ou Bluetooth exigem infraestrutura dedicada e recalibração frequente. Este trabalho propõe e implementa um sistema de localização indoor que estima a posição do usuário sobre a planta baixa a partir de uma **fotografia do teto** combinada com uma leitura de GPS de baixa precisão. A hipótese central é que o teto constitui uma referência visual estável — pouca oclusão por pessoas, baixa variação temporal e iluminação relativamente constante — adequada ao reconhecimento visual de lugar. O sistema combina um filtro geográfico grosseiro (que reduz o espaço de busca de milhares para dezenas de candidatos), busca por similaridade de *embeddings* de imagem e uma etapa opcional de verificação geométrica, seguida de interpolação ponderada da posição. Apresentamos a arquitetura, a implementação (API em Python/FastAPI, PostgreSQL com pgvector e earthdistance, e três aplicativos web progressivos) e uma avaliação teórica em cenário sintético, na qual o sistema obteve erro mediano de aproximadamente 0,6 m e 100% de acertos dentro de 2 m. Discutimos limitações e delineamos a avaliação prática com dados reais como principal trabalho futuro.

Palavras-chave: localização indoor; reconhecimento visual de lugar; embeddings de imagem; pgvector; PostgreSQL; FastAPI.

1. Introdução

Sistemas de navegação por satélite (GNSS) revolucionaram a localização em ambientes abertos, mas sua precisão cai drasticamente em ambientes internos, onde o sinal é atenuado e sofre múltiplos caminhos. Em locais como shopping centers, aeroportos e hospitais, isso impede aplicações de orientação ao usuário, logística e análise de fluxo. Abordagens baseadas em rádio (Wi-Fi *fingerprinting*, balizas Bluetooth, UWB) alcançam boa precisão, mas dependem de infraestrutura instalada e de recalibração periódica, o que eleva custo e manutenção.

A visão computacional oferece uma alternativa atraente: o ambiente já é rico em informação visual e o dispositivo do próprio usuário (um smartphone) dispõe de câmera e GPS. O reconhecimento visual de lugar (*Visual Place Recognition*, VPR) busca identificar onde uma imagem foi capturada comparando-a com uma base de referência. Contudo, fotos ao nível dos olhos sofrem com oclusão por pessoas, mudanças de vitrines e variações de iluminação.

Este trabalho explora uma direção específica: usar o **teto** como referência. Tetos tendem a ser visualmente estáveis ao longo do tempo, raramente são ocluídos por pessoas e têm iluminação mais uniforme, o que os torna candidatos naturais a âncoras visuais confiáveis para localização.

2. Hipótese e objetivos

Hipótese. Dada uma planta baixa de um local e uma base de fotografias do teto rotuladas com a posição em que foram tiradas, é possível estimar a posição de uma nova fotografia do teto sobre a planta, usando o GPS apenas como filtro grosseiro e a imagem como discriminador fino.

Objetivos.

- Projetar e implementar uma API que receba foto do teto + GPS e devolva a posição (x, y) na planta, com uma medida de confiança.
- Construir o fluxo de mapeamento (coleta rotulada) e de consulta em aplicativos web.
- Definir e medir uma métrica de erro em metros, validando o método em cenário controlado.
- Estabelecer uma base extensível para incorporar modelos de visão mais fortes.

3. Trabalhos relacionados

O reconhecimento visual de lugar evoluiu de descritores locais clássicos — como SIFT [1] e ORB [2], casados e filtrados por RANSAC [3] — para descritores globais aprendidos, como NetVLAD [4], e, mais recentemente, para características auto-supervisionadas de propósito geral como o DINOv2 [5]. Para verificação geométrica, métodos modernos de detecção e casamento de pontos, como SuperPoint [6] e LightGlue [7], superam os pipelines clássicos. O descritor de imagem reduzida (*tiny image*) [8] é uma linha de base simples e eficiente para recuperação por similaridade. Do lado de sistemas, a extensão pgvector [9] habilita busca aproximada por vizinhos em PostgreSQL, e a extensão earthdistance permite consultas geográficas por raio sem a complexidade do PostGIS. Nosso sistema combina essas peças num pipeline híbrido voltado ao caso específico do teto.

4. Arquitetura e método

4.1 Visão geral

O sistema opera em duas fases. No **mapeamento**, um coletor fotografa o teto; a cada foto, o aplicativo captura o GPS e o usuário toca na planta para marcar a posição (x, y). Cada foto é convertida em um vetor (*embedding*) e armazenada com seus metadados. Na **consulta**, uma nova foto + GPS são enviadas à API, que estima a posição.

4.2 Pipeline de localização

A consulta percorre quatro etapas encadeadas:

- **Filtro geográfico.** Seleciona as fotos de mapeamento dentro de um raio do GPS, usando `earth_box` e `earth_distance` (extensões `cube/earthdistance`). Reduz o universo de milhares para dezenas de candidatos.
- **Busca por similaridade.** Calcula o `embedding` da consulta e recupera os N mais próximos por distância de cosseno, com `pgvector`.
- **Verificação geométrica (opcional).** Casa características locais (ORB) entre a consulta e cada candidato e aplica RANSAC; o número de *inliers* reordena os candidatos e reforça a confiança. Ativa quando o componente de visão pesado está instalado.
- **Interpolação.** Estima (x, y) como média ponderada das posições dos melhores candidatos, com pesos proporcionais à similaridade (e aos *inliers*, quando disponíveis).

4.3 Modelo de dados

Três entidades principais: **local** (o ambiente mapeado), **planta** (planta baixa em SVG, com escala em metros por unidade) e **foto** (imagem do teto com latitude, longitude, precisão do GPS, posição `plan_x/plan_y` na planta, o `embedding` e o tipo — mapeamento ou consulta). Dois índices sustentam o desempenho: um índice GiST sobre `ll_to_earth(latitude, longitude)` para o filtro por raio e um índice HNSW sobre o `embedding` para a busca por similaridade.

4.4 Representação por embeddings

O backend padrão emprega um descritor *tiny-image*: a imagem é convertida para tons de cinza, redimensionada para 32×16 (512 valores), achatada e normalizada (subtração da média, para invariância a brilho, e normalização L2). O resultado é um vetor de 512 dimensões compatível com a coluna do banco. Essa escolha mantém o serviço leve (sem dependências pesadas), o que favorece o

empacotamento e a implantação, e é suficiente para validar o pipeline de ponta a ponta. A arquitetura prevê a substituição por um backend forte (por exemplo, DINOv2) sem alterações no restante do sistema.

4.5 Filtro geográfico sem PostGIS

Como o PostgreSQL disponível não inclui PostGIS, o filtro por raio usa as extensões *cube* e *earthdistance*. A função `ll_to_earth` projeta a coordenada em um ponto tridimensional, e `earth_box + earth_distance` implementam a busca por proximidade de forma eficiente com índice GiST. Para o requisito deste trabalho — apenas recortar candidatos por proximidade — essa solução é adequada e evita a operação de um segundo banco.

4.6 Verificação geométrica e interpolação

A verificação geométrica confirma que dois quadros observam de fato a mesma região do teto, descartando coincidências de aparência. A posição final é obtida por média ponderada: candidatos mais semelhantes (e com mais inliers) contribuem mais. A confiança reportada deriva da similaridade do melhor candidato. Quando nenhum candidato sobrevive ao filtro, o sistema responde sem posição, sinalizando incerteza.

5. Implementação

A API foi implementada em **Python** com **FastAPI**, persistência em **PostgreSQL 17** (imagem `pgvector/pgvector`) com as extensões *vector*, *cube* e *earthdistance*, e migrações com Alembic. As imagens das fotos são gravadas em volume persistente. O front-end consiste em três aplicativos web progressivos (PWA) em **Vue 3 + Vite** — administração (cadastro de locais e upload da planta), coletor (captura de foto, GPS e marcação na planta) e teste (consulta e visualização do ponto estimado) — que compartilham módulos de câmera, GPS e cliente HTTP.

A implantação é feita em um cluster **Kubernetes (k3s)**, com Ingress Traefik e a imagem construída e importada diretamente no runtime de contêineres do nó. A integração contínua (GitHub Actions) executa verificação de estilo e testes automatizados a cada alteração, com um banco PostgreSQL efêmero. O desenvolvimento seguiu um fluxo disciplinado de *issue* → ramo → *Pull Request* → revisão → integração.

6. Avaliação experimental

Realizamos uma avaliação teórica controlada para validar o pipeline e o método de medição, antes da coleta de dados reais. O cenário é sintético e reproduzível: cria-se um local com uma planta de 1000×1000 unidades e escala de 0,05 m/unidade (ambiente de aproximadamente 50 m de lado). Mapeiam-se 25 posições em grade, cada uma associada a um padrão visual distinto. Para cada posição, gera-se uma **consulta** a partir da mesma imagem, perturbada com ruído gaussiano ($\sigma = 18$) e variação de brilho (+20), simulando uma recaptura. Mede-se o erro como a distância euclidiana entre a posição estimada e a verdadeira, convertida para metros pela escala.

Métrica	Unidades da planta	Metros
Erro mediano	11,90	0,595
Erro médio	10,76	0,538
Erro máximo	18,93	0,947
Taxa de acerto (≤ 2 m)	—	100%

Tabela 1. Resultados da avaliação sintética ($n = 25$, backend `tiny-image`).

O sistema localizou corretamente todas as consultas dentro de 2 m, com erro mediano de cerca de 0,6 m. O resíduo observado decorre da interpolação ponderada, que mistura o candidato correto com vizinhos

de baixa semelhança. Ressalte-se que o objetivo desta avaliação é verificar a corretude do pipeline e da medição; os números não devem ser extrapolados para condições reais, onde a aparência do teto, reflexos e variação de perspectiva impõem desafios adicionais.

7. Discussão e limitações

- **Backend de embedding.** O descritor tiny-image é sensível a rotação, escala e mudanças fortes de iluminação; padrões de teto repetitivos (corredores) podem confundir-lo.
- **Avaliação sintética.** Os resultados validam o método, não a precisão em campo; não capturam variação de pose da câmera nem mudanças reais do ambiente.
- **Dependência do GPS indoor.** O filtro pressupõe GPS minimamente informativo; em locais profundos sem sinal, o raio precisa ser ampliado ou o filtro relaxado.
- **Georreferência.** Nesta fase, GPS e coordenadas da planta são tratados de forma independente; a conversão entre os dois sistemas fica para uma fase posterior.

8. Trabalhos futuros

- **Avaliação prática com dados reais.** Coletar fotos de teto em um ambiente real (por exemplo, um pavimento de shopping), com posição de referência conhecida, e medir o erro em metros em condições de uso, incluindo variação de horário e de aparelho.
- **Embeddings fortes.** Substituir o tiny-image por descritores aprendidos (DINOv2, NetVLAD) e comparar precisão, latência e custo.
- **Verificação geométrica moderna.** Avaliar SuperPoint + LightGlue contra ORB/RANSAC e medir o ganho na reordenação dos candidatos.
- **Calibração do filtro geográfico.** Ajustar raio, top-N e limiares por local, em função da densidade de mapeamento e da qualidade do GPS.
- **Georreferência da planta (fase 2).** Ancorar a planta ao mundo real com pontos de controle, permitindo converter entre GPS e coordenadas da planta.
- **Robustez e operação.** Exposição externa segura (túnel), monitoramento de qualidade das estimativas e estratégias de reindexação incremental em larga escala.

9. Conclusão

Apresentamos um sistema completo e funcional de localização indoor baseado em imagens do teto, do modelo de dados à API, aos aplicativos de coleta e teste e à implantação em Kubernetes. A avaliação teórica confirmou a corretude do pipeline e do método de medição, com erro mediano sub-métrico em cenário controlado. O resultado sustenta a hipótese de que o teto é uma referência visual conveniente e motiva a próxima etapa: a avaliação prática com dados reais e a incorporação de modelos de visão mais robustos.

Referências

- [1] Lowe, D. G. Distinctive image features from scale-invariant keypoints. International Journal of Computer Vision, 2004.
- [2] Rublee, E.; Rabaud, V.; Konolige, K.; Bradski, G. ORB: An efficient alternative to SIFT or SURF. ICCV, 2011.
- [3] Fischler, M. A.; Bolles, R. C. Random sample consensus (RANSAC). Communications of the ACM, 1981.
- [4] Arandjelović, R. et al. NetVLAD: CNN architecture for weakly supervised place recognition. CVPR, 2016.
- [5] Oquab, M. et al. DINOv2: Learning robust visual features without supervision. arXiv:2304.07193, 2023.
- [6] DeTone, D.; Malisiewicz, T.; Rabinovich, A. SuperPoint: Self-supervised interest point detection and description. CVPR Workshops, 2018.
- [7] Lindenberger, P.; Sarlin, P.-E.; Pollefeys, M. LightGlue: Local feature matching at light speed. ICCV, 2023.
- [8] Torralba, A.; Fergus, R.; Freeman, W. T. 80 million tiny images. IEEE TPAMI, 2008.
- [9] pgvector: Open-source vector similarity search for PostgreSQL. <https://github.com/pgvector/pgvector>.